技术部第五次技术分享会

枚举、泛型与常用标准库

姚雨森　技术部分管主席

用整数表示状态，需要记住编号的含义

一个订单可能处于待付款、已付款、已发货或已取消状态。

```
int status = 0;

if (status == 1) {
    System.out.println("Ready to ship");
}

status = 7;
```

- 阅读代码时，需要另外查找：0 和 1 分别代表什么？
- int 允许许多整数，但业务只需要四种状态。
- 即使 7 没有业务含义，赋值语句仍能通过类型检查。

换成字符串更直观，但仍然可能写错

```
String status = "PENDING";
status = "PAID";

status = "PAIID";
status = "paid";
```

- 名称比编号容易理解，但任何字符串都可以赋给 status。
- 拼写错误、大小写不一致，都可能让后面的判断失效。
- 我们希望同时表达每个选项的含义和允许使用的选项。

枚举：为有限的一组选择定义类型

订单状态只需要以下四个选项：

含义	枚举常量名称
待付款	PENDING
已付款	PAID
已发货	SHIPPED
已取消	CANCELLED

枚举（enum）将这些选项列在一起，定义一个专门的类型。

星期、方向、任务状态等有限选项，也可以用枚举表达。

使用 enum 定义订单状态类型

将下面的定义放在 OrderStatus.java 中：

```
public enum OrderStatus {
    PENDING,
    PAID,
    SHIPPED,
    CANCELLED
}
```

- enum 用于声明枚举；OrderStatus 是新类型的名称。
- 花括号中列出枚举常量，使用逗号分隔。
- 类型名称通常使用大驼峰；常量通常使用大写字母和下划线。
- 使用常量时写成类型名. 常量名，例如 OrderStatus.PAID。

像其他类型一样声明变量和赋值

以下语句写在 Main.java 的 main 方法中，两个源文件放在同一目录。

```
OrderStatus status = OrderStatus.PENDING;
System.out.println(status); // PENDING

status = OrderStatus.PAID;
System.out.println(status); // PAID
```

- 左边的 OrderStatus 指定变量类型。
- 右边的 OrderStatus.PENDING 是该类型的一个常量。
- 变量可以重新赋值；这里是改为保存另一个枚举常量。

类型检查帮助我们排除无效的赋值

下面的错误写法用于说明类型限制，不能通过编译：

```
OrderStatus status = OrderStatus.PENDING;

status = 1;                    // Wrong type
status = "PAID";               // Wrong type
status = OrderStatus.PAIID;    // No such constant
```

- 枚举常量有自己的类型；字符串 "PAID" 并不是它。
- 拼错常量名称时，编译器就能指出问题。
- 另一种枚举类型的常量，即使同名，也不能赋给这个变量。

补充：枚举变量也可以为 null，表示没有引用枚举常量；本例始终给它赋明确的状态。

使用 == 判断是否为某个枚举常量

```
OrderStatus status = OrderStatus.PAID;

if (status == OrderStatus.PAID) {
    System.out.println("Ready to ship");
}

if (status != OrderStatus.CANCELLED) {
    System.out.println("Order is not cancelled");
}
```

- == 判断是否为同一个枚举常量；!= 判断是否不同。
- 直接比较枚举值，不需要先转换成编号或字符串。
- 字符串内容比较仍使用 equals，不要把这里的规则照搬过去。

枚举作为参数：明确方法接收哪一种状态

假设本例规定：只有待付款的订单可以取消。

在 Main 类中定义方法：

```
static boolean canCancel(OrderStatus status) {
    return status == OrderStatus.PENDING;
}
```

在 main 方法中调用：

```
boolean first = canCancel(OrderStatus.PENDING); // true
boolean second = canCancel(OrderStatus.SHIPPED); // false
```

参数的类型说明需要什么数据；方法体决定这组数据对应的业务规则。